

Question Bank — Week 14

Graphs and Graph Searches (Lecture 14) — Questions

DSA27 · Data Structures and Algorithms

Fall 2026

Contents

Part A — Multiple choice	2
Part B — Short answer and essay	5
Part C — Trace the algorithms	5
Part D — Graph state	6
Part E — Complexity analysis	6
Part F — Find and fix the bug	7
Part G — Write the code	8

Answers are in a separate file: [week14-answers.md](#). Levels: **[what]** recall · **[how]** apply · **[why]** explain and justify. The functions are those of `dsa/graph.py`. `Graph` is an adjacency list that keeps each node's neighbours in the order the edges were added, and returns `nodes()` in the order the nodes were added; `MatrixGraph` is an adjacency matrix. `bfs` marks a node when it is **enqueued**; `dfs` is recursive; `dfs_iterative` marks a node when it is **popped** and pushes neighbours in reverse; `topological_sort` is Kahn's algorithm with a queue. Unless a question says otherwise, **H** is the undirected graph built by

A-B, A-D, B-C, B-E, D-E, C-F, E-F, F-G

(added in that order), and **P** is the directed graph built by

A->C, B->C, B->D, C->E, D->E, E->F.

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W14-M01 – W14-M22
B	Short answer and essay	W14-E1 – W14-E5
C	Trace the algorithms	W14-T1 – W14-T5
D	Graph state — draw every step	W14-S1 – W14-S3
E	Complexity analysis	W14-K1 – W14-K3
F	Find and fix the bug	W14-B1 – W14-B4

Part	Type	Questions
G	Write the code — checked by pytest	W14-C1 – W14-C5

Part A — Multiple choice

W14-M01 [what] An adjacency matrix for a graph with V vertices and E edges uses memory:

- **a)** $O(V + E)$
- **b)** $O(E)$
- **c)** $O(V^2)$
- **d)** $O(V \log V)$

W14-M02 [what] Breadth-first search keeps the vertices waiting to be expanded in a:

- **a)** queue
- **b)** stack
- **c)** priority queue
- **d)** sorted array

W14-M03 [how] An undirected graph has 7 vertices with degrees 2, 3, 2, 2, 3, 3 and 1. How many edges does it have?

- **a)** 16
- **b)** 7
- **c)** 14
- **d)** 8

W14-M04 [why] BFS from s always reaches a vertex first along a path with the fewest edges because:

- **a)** the queue releases vertices in order of non-decreasing distance from s
- **b)** it marks vertices when they are dequeued
- **c)** it tries every path and keeps the shortest
- **d)** the adjacency lists are sorted

W14-M05 [how] `bfs(H, "A")` returns:

- **a)** A B C D E F G
- **b)** A B D C E F G
- **c)** A B C F E D G
- **d)** A D B E C F G

W14-M06 [how] `dfs(H, "A")` returns:

- **a)** A B D C E F G
- **b)** A B C E F D G
- **c)** A B C F E D G
- **d)** A D E B C F G

W14-M07 [how] `shortest_path_unweighted(H, "D", "C")` returns:

- **a)** ["D", "E", "F", "C"]
- **b)** ["D", "A", "B", "C"]
- **c)** ["D", "E", "B", "C"]
- **d)** None

W14-M08 [what] `has_edge(u, v)` on an adjacency matrix costs:

- **a)** $O(V)$
- **b)** $O(\deg u)$
- **c)** $O(E)$
- **d)** $O(1)$

W14-M09 [what] `neighbours(u)` on an adjacency matrix costs:

- **a)** $O(V)$
- **b)** $O(1)$
- **c)** $O(\deg u)$
- **d)** $O(E)$

W14-M10 [why] A BFS on a `MatrixGraph` costs $O(V^2)$ even when the graph has very few edges, because:

- **a)** the queue has to grow
- **b)** finding each vertex's neighbours scans its whole row of V cells
- **c)** every cell of the matrix is hashed
- **d)** the visited set is a matrix too

W14-M11 [how] An undirected graph with E edges is stored as an adjacency list. How many neighbour entries do the lists hold in total?

- **a)** E
- **b)** $2E$
- **c)** E^2
- **d)** V^2

W14-M12 [why] BFS marks a vertex as visited when it is **enqueued**, not when it is dequeued, because otherwise:

- **a)** a vertex can be put in the queue more than once
- **b)** BFS would miss some reachable vertices
- **c)** the queue would raise `IndexError`
- **d)** the search would no longer find shortest paths

W14-M13 [why] `dfs_iterative` pushes each vertex's neighbours in **reverse** order because:

- **a)** a stack is first in, first out
- **b)** the first neighbour must come off the stack first, to match `dfs`
- **c)** otherwise the search never ends on a cycle
- **d)** it saves memory

W14-M14 [why] The recursive dfs is run on a graph that is one path of 5,000 vertices. In Python, with the default settings, it:

- **a)** returns all 5,000 vertices
- **b)** runs out of memory
- **c)** raises RecursionError
- **d)** loops for ever

W14-M15 [what] A DAG is:

- **a)** a connected undirected graph with no cycle
- **b)** a graph whose every vertex has the same degree
- **c)** a graph stored as a matrix
- **d)** a directed graph with no cycle

W14-M16 [how] `topological_sort` on the directed graph $A \rightarrow B, B \rightarrow C, C \rightarrow A$ returns:

- **a)** ["A", "B", "C"]
- **b)** []
- **c)** None
- **d)** it raises ValueError

W14-M17 [why] Undirected cycle detection ignores a visited neighbour that is the vertex's parent, because:

- **a)** every undirected edge is stored in both lists, so the vertex you came from always looks visited
- **b)** the parent is always grey
- **c)** a parent cannot be part of a cycle
- **d)** it makes the search $O(V + E)$

W14-M18 [why] In the directed diamond $A \rightarrow B, A \rightarrow C, B \rightarrow D, C \rightarrow D$, a DFS from A meets D a second time, from C. That is not a cycle, because at that moment D is:

- **a)** white
- **b)** grey
- **c)** black
- **d)** the start vertex

W14-M19 [how] A graph has vertices A to I and the undirected edges A-B, B-C, D-E, F-G. How many connected components does it have?

- **a)** 4
- **b)** 3
- **c)** 9
- **d)** 5

W14-M20 [how] `topological_sort(P)` returns:

- **a)** A B C D E F
- **b)** B D A C E F
- **c)** A C B D E F

- **d)** F E D C B A

W14-M21 [why] Which of these can BFS on its own **not** answer correctly?

- **a)** the path with the fewest edges from s to t
- **b)** whether t can be reached from s
- **c)** the shortest route by kilometres on a road map with different road lengths
- **d)** the connected components of an undirected graph

W14-M22 [how] One BFS over an adjacency list costs:

- **a)** $O(V \cdot E)$
 - **b)** $O(V^2)$
 - **c)** $O(E \log V)$
 - **d)** $O(V + E)$
-

Part B — Short answer and essay

W14-E1 [why] (4 marks) Compare the adjacency list and the adjacency matrix: memory, `has_edge`, `neighbours`, and the cost of one BFS. Which would you choose for a road map of 100,000 junctions, and which for a graph of 500 people in which almost everyone knows almost everyone and you ask “do these two know each other?” millions of times? Justify with numbers.

W14-E2 [why] (4 marks) Describe breadth-first search. Explain why it finds shortest paths by number of edges, how `shortest_path_unweighted` rebuilds the path, and why the answer can be wrong on a road map with different road lengths.

W14-E3 [why] (4 marks) Recursive DFS and DFS with your own stack: what does the call stack remember for you? Why must `dfs_iterative` mark a vertex when it is **popped** and push neighbours in **reverse** to give the same order as `dfs`? Why write the iterative version at all in Python?

W14-E4 [why] (4 marks) Explain cycle detection in an undirected graph and in a directed graph. Why does each rule give wrong answers when used on the other kind of graph? Give a small example of each failure.

W14-E5 [why] (3 marks) Define a topological order. When does one exist? Describe Kahn’s algorithm, how it detects that no order exists, and one real application.

Part C — Trace the algorithms

W14-T1 [how] Trace `bfs(H, "A")`: after each vertex is taken off the queue, give the vertices it adds and the queue. What is the distance from A to each vertex?

W14-T2 [how] Trace the recursive `dfs(H, "A")`: show each call, indented by depth, and each neighbour that is skipped because it is already visited. What is the deepest the call stack gets?

W14-T3 [how] Trace `dfs_iterative(H, "A")`: after each pop, the action and the stack (bottom $\rightarrow$ top). Which vertices are pushed twice?

W14-T4 [how] Trace `shortest_path_unweighted(H, "A", "G")`: give the parent map as BFS builds it, and the walk back from G. Then give `shortest_path_unweighted(H, "D", "C")`, and explain why a different path of the same length is not returned.

W14-T5 [how] Trace `topological_sort(P)`: the in-degrees at the start, and after each vertex is taken, the in-degrees that change and the queue. Is B D A C E F also a valid topological order of P?

Part D — Graph state

W14-S1 [how] Starting from an empty undirected Graph and an empty MatrixGraph, apply to both:

```
add_edge A B, add_edge B C, add_node D, add_edge C A, add_edge D B
```

Draw the adjacency lists after each operation, and the matrix at the end, with rows and columns in node order. Then give `nodes()`, `edges()` and `degree("B")`.

W14-S2 [how] The directed graph Q is built by

```
A->B, B->C, A->D, D->C, D->E, E->F, F->D
```

Run `has_cycle(Q)`. After every vertex is entered or finished, give the colour of every vertex seen so far (white, grey, black). Which edge proves the cycle? Which edge reaches a black vertex, and why is that not a cycle? What would `topological_sort` return for Q, and for Q without the edge $F \rightarrow D$?

W14-S3 [how] The undirected graph U is built by A-B, A-C, C-D, D-E, E-C. Trace `has_cycle(U)` with the parent check: for each call, the vertex, its parent, and what happens to each neighbour. At which moment is the cycle found?

Part E — Complexity analysis

W14-K1 [how] Give Θ for this code on a Graph with V vertices and E edges, and on a MatrixGraph, and say what it computes for an undirected graph:

```
total = 0
for node in graph.nodes():
```

```
for other in graph.neighbours(node):  
    total += 1
```

W14-K2 [why] A student writes connected components without the seen check:

```
def components(graph):  
    return [bfs(graph, node) for node in graph.nodes()]
```

What is wrong with the result, and what is its cost? What is the cost of the correct version, and why?

W14-K3 [why] To count the edges of an undirected graph, a student asks about every pair:

```
count = 0  
for u in graph.nodes():  
    for v in graph.nodes():  
        if graph.has_edge(u, v):  
            count += 1  
count //= 2
```

Give Θ on a Graph and on a MatrixGraph. What does `len(graph.edges())` cost on each?

Part F — Find and fix the bug

W14-B1 [how]

```
def bfs(graph, start):  
    order = []  
    visited = ChainingHashMap()  
    queue = CircularQueue()  
    queue.enqueue(start)  
    while not queue.is_empty():  
        node = queue.dequeue()  
        visited.put(node, True)  
        order.append(node)  
        for neighbour in graph.neighbours(node):  
            if neighbour not in visited:  
                queue.enqueue(neighbour)  
    return order
```

W14-B2 [why] In Graph:

```
def add_node(self, node):  
    self._adjacent.put(node, DynamicArray())  
    self._order.append(node)
```

W14-B3 [why] Undirected cycle detection:

```
def has_cycle(graph):
    visited = ChainingHashMap()

    def visit(node):
        visited.put(node, True)
        for neighbour in graph.neighbours(node):
            if neighbour in visited:
                return True
            if visit(neighbour):
                return True
        return False

    for node in graph.nodes():
        if node not in visited and visit(node):
            return True
    return False
```

W14-B4 [why] The same function as W14-B3, used as the **directed** cycle check. Find an acyclic directed graph on which it returns True, and fix it.

Part G — Write the code

In `practice/week14.py`; check with `pytest tests/test_practice_week14.py -v`. Use your own `Graph`, `ChainingHashMap`, `CircularQueue` or `Stack` as working storage — not a Python dict, set or deque.

W14-C1 [how] `maze_distance(maze)` — the fewest steps from S to T in a grid maze, or \$-\$1.

W14-C2 [why] `is_bipartite(graph)` — can the vertices be coloured with two colours so that every edge joins different colours? Why does an odd cycle make it impossible?

W14-C3 [how] `topological_order_dfs(graph)` — a topological order from reversed DFS finishing order, or None for a cycle.

W14-C4 [why] `semester_plan(prerequisites)` — the courses grouped into the fewest semesters. Why is the number of semesters the length of the longest prerequisite chain?

W14-C5 [how] `count_islands(grid)` — connected components of the land cells in a grid.